```
<html>
<head>
    <title>My First Tauri Application</title>
</head>
<body>
    <h1>Hello OSFY readers!</h1>
    Message from Rust: <span id "message"></span>
    <script>
        async function callRust() {
            const { invoke } = window.__TAURI__.tauri
            var output = document.getElementById("message");
            output.innerHTML = await invoke('send_message_to_ui');
        }
    </script>
    <div>
        <input type="button" value "Click me" onclick="callRust();"
    </div>
</body>
</html>
```

Figure 4: Code changes to *index.html*

```
// Prevents additional console window on windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

#[tauri::command]
fn send_message_to_ui() -> &'static str {
    "Message from Rust!"
}

fn main() {
  tauri::Builder::default()
    .invoke_handler(tauri::generate_handler![send_message_to_ui])
    .run(tauri::generate_context!())
    .expect("error while running tauri application");
}
```

Figure 5: Changes to *main.rs*

```
{
  "build": {
    "beforeBuildCommand": "",
    "beforeDevCommand": "",
    "devPath": "../frontend",
    "distDir": "../frontend",
    "withGlobalTauri": true
  },
```

Figure 6: Changes to Tauri config

Figure 7: Before clicking the button

Figure 8: After clicking the button

Size:	3.60 MB (37.78.560 bytes)
Size on disk:	3.60 MB (37.80.608 bytes)

Figure 9: Tauri executable size

This means they can be accessed from any part of the application code without requiring an import statement for each method. This can be avoided if you are using frameworks like React, but we are simply using HTML and pure JS. Tauri CLI will make note of these changes once we save, and recompile and reload the application with our new changes.

Bundling our application to an executable

Bundling or compiling our application to an executable format like '.exe' is rather simple. Just use the command:

```
cargo tauri build
```

This will create an installer for your application if you are on Windows. Here is the amazing part of Tauri — the size of this application is just under 3MB!

The same application with Electron would have been at least 30MB or more because of Chromium's renderer and entire Node.js bundled within it. Tauri needs none of that and also uses the OS to handle the backend work, which makes it more performant as well.

So, we have seen that Tauri has the potential to become the future of UI development. Its ability to create lightweight, fast, and secure applications, while providing cross-platform compatibility and a variety of language choices, makes it a promising option for developers looking to streamline their workflow and improve user experience. I have only scratched the surface of Tauri's capabilities here. Readers who wish to read more about Tauri's ongoing development and its features can visit *https://tauri.app/*. END

By: S. Sreyas

The author is a highly skilled programmer who has gained considerable expertise in the field through self-learning. He has made significant contributions to various open source projects, with a particular focus on reverse engineering, compiler design, and system level programming. His contributions are available on GitHub.